Python 程式語言

Lecture 2: Basic Syntax and Flow Control

Instructor: 顏家珩 Chia-Heng Yen

Course Roadmap

- Fundamentals & Environment
  - Introduction to Python and Development Environment Setup
  - Basic Syntax and Flow Control
  - Data Structures and Collections
- Advanced Programming
  - Modular and Functional Programming
  - File Handling and Exception Handling
  - Object-Oriented Programming
  - Algorithm Implementation in Python
  - Modules and Package Applications
- User Interface Development
  - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
  - Data Analysis Fundamentals
  - Scientific Computing Applications
  - Data Visualization
- AI & ML Applications
  - Introduction to Machine Learning with Applications
  - AI-Related Library in Python Applications

Week 1 Recap

- Course introduction
- Development environment setup
- Git version control basics

- HW1: First Python programs

Outline

- Master Python syntax fundamentals
  - Indentation (縮排) rules and best practices
  - Variable naming conventions (慣例)
  - PEP 8 coding standards
- Understand data types and operations
  - Numeric operations and precision
  - String manipulation techniques
  - Type conversion and validation
- Implement decision-making logic
  - Conditional statements (if/elif/else)
  - Boolean logic and comparisons
  - Complex conditional expressions
- Create repetitive task automation
  - for loops for known iterations
  - while loops for conditional repetition
  - Loop control with break/continue

Python Syntax Features

Design Philosophy

- "The Zen of Python" by Tim Peters
  - Beautiful is better than ugly
    美麗勝過醜陋
  - Explicit is better than implicit
    明確勝過隱含
  - Simple is better than complex
    簡單勝過複雜
  - Flat is better than nested
    扁平結構優於巢狀結構
  - Sparse is better than dense
    稀疏優於密集
  - Readability counts
    可讀性很重要
  - There should be one (and preferably only one) obvious way to do it
    應該有一種（最好只有一種）明顯的方法來做

Python Syntax Features

- Indentation-Based Structure

```
# Python enforces clean code structure
if student_age >= 18:
        print("Eligible for university admission")
        if gpa >= 3.0:
                print("Meets GPA requirements")
                can_apply = True
        else:
                print("GPA too low")
                can_apply = False
else:
        print("Too young for university")

# Compare with other languages:
# C/Java: if (condition) { ... }
# Pascal: if condition then begin ... End
# Python: if condition:  (clean and readable!)
```

Python Syntax Features

- Case Sensitivity

```
# Everything in Python is case-sensitive
student_name = "Alice"
Student_Name = "Bob" # Different variable!
STUDENT_NAME = "Charlie" # Also different!

# Function names
print("Hello") # Built-in function
Print("Hello") # NameError!

# Keywords
if True: # Correct
        pass
If True: # SyntaxError!
```

Python Syntax Features

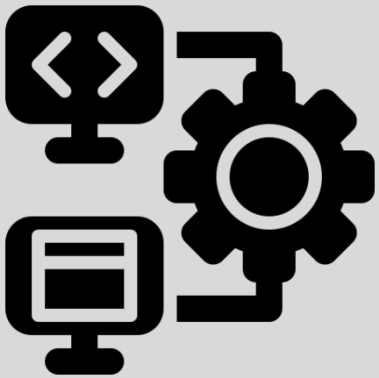
- Dynamic Typing

```
# Variables can change type during runtime
grade = 95 # Integer
print(f"Grade: {grade}, Type: {type(grade)}")

grade = "A" # Now it's a string
print(f"Grade: {grade}, Type: {type(grade)}")

grade = [85, 90, 88] # Now it's a list
print(f"Grade: {grade}, Type: {type(grade)}")

# This flexibility is powerful but requires careful coding
```

Python Syntax Features

- Interactive and Interpreted

```
# No compilation step required
# Write code → Run immediately
# Perfect for learning and experimentation


# REPL (Read-Eval-Print Loop) Example:
# >>> name = "Python“
# >>> print(f"I love {name}!")
# I love Python!
# >>> 2 + 3 * 4
# 14
```



Python Syntax Features

- Rich Built-in Types

```
# Powerful built-in data structures
numbers = [1, 2, 3, 4, 5]                          # List (串列)
coordinates = (10, 20)                             # Tuple (多元組)
student_info = {"name": "Alice", "age": 20}        # Dictionary (字典)
unique_grades = {85, 90, 88, 95}                   # Set (集合)
message = "Hello, Python World!"                   # String (字串)

# Each type has rich methods and operations
```

Python Syntax Features

- Extensive Standard Library

```
# "Batteries included" philosophy
import datetime
import random
import math
import os


# Rich ecosystem of third-party packages
# pip install requests pandas matplotlib numpy
```

- “Batteries included (電池已包含)” philosophy
  - A software design principle that means a programming language or software package comes with a comprehensive standard library and built-in tools, allowing users to accomplish most common tasks without needing to install additional third-party packages

Python Basic Syntax

Indentation Rules

- Python uses indentation to define code blocks

```
# Correct 正確
if True:
        print("This is indented")
        print("This is also indented")
# Incorrect 錯誤
if True:
print("This will cause an error")
```

- Key Points
  - Use 4 spaces per indentation level
  - Be consistent throughout your code
  - Avoid mixing tabs and spaces

Understanding Python Indentation

- Code Structure Definition: Python uses indentation to define code blocks instead of braces {} or keywords

```
# Other languages use braces
# if (condition) {
#       statement1;
#       statement2;
# }

# Python uses indentation
if condition:
        statement1
        statement2
```

Common Indentation Errors

- Error 1: No Indentation

```
score = 85
if score >= 80:
print("Good grade!")
print("Well done!")
# IndentationError: expected an indented block

# Correct version
score = 85
if score >= 80:
        print("Good grade!")          # 4 spaces indentation
        print("Well done!")           # Same level indentation
```

Common Indentation Errors

- Error 2: Inconsistent Indentation

```
# Inconsistent indentation levels
if temperature > 30:
    print("Very hot")          # 4 spaces
        print("Stay cool!")    # 8 spaces - IndentationError!
    print("Drink water")       # 4 spaces

# Consistent indentation
if temperature > 30:
    print("Very hot")          # 4 spaces
    print("Stay cool!")        # 4 spaces
    print("Drink water")       # 4 spaces
```

Common Indentation Errors

- Error 3: Mixed Tabs and Spaces

```
# Never mix tabs and spaces
if weather == "rainy":
    print("Take umbrella")        # 4 spaces
  print("Wear raincoat")          # 1 tab - IndentationError!
    print("Stay dry")             # 4 spaces

# Use only spaces (recommended)
if weather == "rainy":
    print("Take umbrella")        # 4 spaces
    print("Wear raincoat")        # 4 spaces
    print("Stay dry")             # 4 spaces
```

Common Indentation Errors

- Error 4: Incorrect Nesting

```
# Wrong indentation levels for nested structures
if age >= 18:
    if has_license:
        print("Can drive")
    print("Is adult")          # This belongs to outer if
  print("End of check")        # Wrong indentation level

# Correct nested structure
if age >= 18:
    if has_license:
        print("Can drive")     # 8 spaces (nested)
        print("Be careful")    # 8 spaces (same level)
    print("Is adult")          # 4 spaces (outer if)
print("End of check")          # 0 spaces (main level)
```

Additional Common Mistakes

```
# Forgetting colon before indented block
if score > 90                    # Missing colon
    print("Excellent")
# Indenting when not needed
print("Hello")
    print("World")               # IndentationError: unexpected indent
# Wrong indentation after function definition
def calculate_grade():
print("Calculating...")          # IndentationError: expected an indented block

# Correct versions
if score > 90:                   # Colon added
    print("Excellent")

print("Hello")
print("World")                   # Same level as previous print

def calculate_grade():
    print("Calculating...")      # Proper indentation
    return grade
```

PEP 8: Python Style Guide

- What is PEP 8? Official Python Style Guide
  - PEP = Python Enhancement Proposal
  - PEP 8 = Style Guide for Python Code
    - Written by Guido van Rossum and others
    - Published in 2001, regularly updated

- Why Follow PEP 8?
  - Consistency (一致性): Code looks the same across different projects
  - Readability (可讀性): Easier to understand and maintain code
  - Collaboration (協作): Team members can easily read each other's code
  - Professional (專業): Industry standard for Python development
  - Tool Support (工具支援): IDEs and linters can check PEP 8 compliance

Key Rules and Examples in PEP 8

1. Line Length
   - Maximum 79 characters per line
   - For comments and docstrings: 72 characters
2. Naming Conventions
   - snake_case for Variables and Functions …
3. Whitespace and Spacing
   - Proper spacing around operators:
     grade_total = midterm_score + final_score + homework_score
   - Function calls and definitions:
     student_grade = calculate_letter_grade(87, curve_points=3)
     final_grades = [calculate_letter_grade(score) for score in test_scores]
   - Collections: proper spacing
     student_scores = [85, 92, 78, 96, 88]
   - Slicing: no spaces around colons
     first_half_scores = student_scores[:3]
   - Keyword arguments
     def enroll_student(student_id, course_code, semester='Fall', year=2024):
4. Import Organization: Standard library, Third-party, Local imports
5. Comments and Documentation: “””…”””, # …, TODO: …, # FIXME:
6. Function and Class Structure

Automated PEP 8 Checking Tools

1. pylint - Comprehensive code analysis
2. flake8 - Fast style guide enforcement
3. black - Automatic code formatting
4. autopep8 - Automatic PEP 8 fixing
5. IDE Integration
   - VS Code: Python extension includes linting
   - PyCharm: Built-in PEP 8 inspection
   - Most IDEs can show PEP 8 violations in real-time

PEP 8 in Practice

- Remember: PEP 8 is a guide, not absolute rules
  - "A foolish consistency is the hobgoblin of little minds“

- When to break PEP 8:
  1. When following the rule would make code less readable
  2. To be consistent with existing code that breaks PEP 8
  3. When backward compatibility is required
  4. In cases where the rule simply doesn't make sense

- But for learning: Follow PEP 8 strictly

PEP 8 Coding Standards

- Python Enhancement Proposal 8
  - Naming Conventions
    - Variables: snake_case
    - Functions: snake_case
    - Classes: PascalCase
    - Constants: UPPER_CASE

```
# Good examples
user_name = "John"
def calculate_total():
        pass
class StudentRecord:
        pass
MAX_SIZE = 100
# Bad examples
userName = "John"      # camelCase
Calculate_Total = ""   # Mixed case
```

Variable Naming Rules

- Rules
  1. Must start with letter or underscore
  2. Can contain letters, numbers, underscores
  3. Case sensitive
  4. Cannot use Python keywords

```
# Valid names
name = "Alice"
_private = 42
user_1 = "Bob"
AGE = 25

# Invalid names
# 2name = "Error"          # Starts with number
# user-name = "Error"      # Contains hyphen
# class = "Error"          # Python keyword
```

Reserved Keywords

- What are Reserved Keywords?
  - Reserved keywords are special words that have predefined meanings in Python and cannot be used as variable names, function names, or identifiers

```
import keyword
print("Python Keywords:")
print(keyword.kwlist)


# Output will show all 35 keywords:
# ['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',
# 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except',
# 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is',
# 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return',
# 'try', 'while', 'with', 'yield']
```

Data Types Review

- Basic Data Types

```
# Numeric Types
integer_num = 42          # int
float_num = 3.14          # float
complex_num = 3 + 4j      # complex
# Text Type
text = "Hello World"      # str
# Boolean Type
is_active = True          # bool
 is_inactive = False

# Check data type
print(type(integer_num)) # <class 'int'>
print(type(text))        # <class 'str'>
```

Operators : Arithmetic Operator

```
a = 10
b = 3

print(a + b)        # Addition: 13
print(a - b)        # Subtraction: 7
print(a * b)        # Multiplication: 30
print(a / b)        # Division: 3.333...
print(a // b)       # Floor Division: 3
print(a % b)        # Modulus: 1
print(a ** b)       # Exponentiation: 1000
```

Operators: Comparison Operators

```
x = 5
y = 10

print(x == y)          # Equal: False
print(x != y)          # Not equal: True
print(x < y)           # Less than: True
print(x > y)           # Greater than: False
print(x <= y)          # Less than or equal: True
print(x >= y)          # Greater than or equal: False

# String comparison
print("apple" < "banana") # True (alphabetical)
```

Operators: Logical Operators

```
a = True
b = False

print(a and b)          # AND: False
print(a or b)           # OR: True
print(not a)            # NOT: False

# Practical example
age = 25
has_license = True

can_drive = age >= 18 and has_license
print(can_drive)        # True
```

Operators: Bitwise Operators

```
a = 10                  # Binary: 1010
b = 4                   # Binary: 0100

print(a & b)            # AND: 0 (0000)
print(a | b)            # OR: 14 (1110)
print(a ^ b)            # XOR: 14 (1110)
print(~a)               # NOT: -11
print(a << 1)           # Left shift: 20
print(a >> 1)           # Right shift: 5
```

Type Conversion: Implicit (隱式) Conversion

```
# Python automatically converts types

integer_num = 10
float_num = 3.14
result = integer_num + float_num        # int + float = float
print(result)                           # 13.14
print(type(result))                     # <class 'float'>
```

Type Conversion: Explicit Conversion

```
# Manual type conversion

str_num = "123"
int_num = int(str_num)          # String to integer
float_num = float(str_num)      # String to float
bool_val = bool(int_num)        # Integer to boolean

print(int_num)                  # 123
print(float_num)                # 123.0
print(bool_val)                 # True
```

Type Checking

```
# Using type() function
data = 42
print(type(data))                    # <class 'int'>
print(type(data) == int)             # True

# Using isinstance() function
print(isinstance(data, int))         # True
print(isinstance(data, str))         # False

# Check multiple types
print(isinstance(data, (int, float)))  # True
```

Conditional Control Structures

if Statement

- Basic Syntax

```
if condition:
        # Code block executes if condition is True
        statement1
        statement2
```

- Example

```
temperature = 25

if temperature > 20:
        print("It's warm today!")
        print("Good weather for outdoor activities")

# Output: It's warm today!
#         Good weather for outdoor activities
```

if-else Statement

```
age = 17
if age >= 18:
        print("You are an adult")
        print("You can vote")
else:
        print("You are a minor")
        print("You cannot vote yet")

# Output: You are a minor
#             You cannot vote yet
```

if-elif-else Statement

```
score = 85

if score >= 90:
        grade = "A"
        print("Excellent!")
elif score >= 80:
        grade = "B"
        print("Good job!")
elif score >= 70:
        grade = "C"
        print("Average")
elif score >= 60:
        grade = "D"
        print("Below average")
else:
        grade = "F"
        print("Failed")

print(f"Your grade is: {grade}")
# Output : Good job!
#                Your grade is: B
```

Nested Conditional Statements

```
weather = "sunny"
temperature = 25

if weather == "sunny":
        print("It's a sunny day!")
        if temperature > 30:
                print("It's hot! Stay hydrated.")
        elif temperature > 20:
                print("Perfect weather for a walk!")
        else:
                print("A bit cool, but still nice!")
else:
        print("Not a sunny day.")
        if temperature < 10:
                print("It's quite cold!")
```

```
# Output : It's a sunny day!
#          Perfect weather for a walk!
```

Ternary Operator

- Syntax

```
result = value_if_true if condition else value_if_false
```

- Examples

```
# Traditional if-else
age = 20
if age >= 18:
        status = "adult"
else:
        status = "minor"
```

```
# Ternary operator
status = "adult" if age >= 18 else "minor"


# More examples
max_value = a if a > b else b
message = "Pass" if score >= 60 else "Fail"
```

Short-Circuit Evaluation

- AND Operator (and)

```
# Second condition won't be evaluated if first is False
x = 0
result = (x != 0) and (10 / x > 1)        # Safe from division by zero
print(result)                             # False

# Both conditions evaluated
x = 5
result = (x != 0) and (10 / x > 1)        # 10/5 > 1 is True
print(result)                             # True
```

Short-Circuit Evaluation

- OR Operator (or)

```
# Second condition won't be evaluated if first is True
user_input = ""
if user_input or input("Enter something: "):
        print("Got input!")
```

Loop Control Structures

for Loop

- Basic Syntax

```
for item in iterable:
        # Code block
        statement
```

- Examples

```
# Iterate over a list
fruits = ["apple", "banana", "orange"]
for fruit in fruits:
        print(f"I like {fruit}")

# Iterate over a string
word = "Python"
for letter in word:
        print(letter)
```

range() Function

- Syntax

```
range(stop)                 # 0 to stop-1
range(start, stop)          # start to stop-1
range(start, stop, step)    # start to stop-1 with step
```

- Examples

```
# Basic range
for i in range(5):
    print(i) # Prints 0, 1, 2, 3, 4

# Range with start and stop
for i in range(2, 8):
    print(i) # Prints 2, 3, 4, 5, 6, 7
```

```
# Range with step
for i in range(0, 10, 2):
    print(i) # Prints 0, 2, 4, 6, 8

# Reverse range
for i in range(10, 0, -1):
    print(i) # Prints 10, 9, 8, ..., 1
```

while Loop

- Basic Syntax

```
while condition:
        # Code block executes while condition is True
        statement
```

- Examples

```
# Simple while loop
count = 0
while count < 5:
        print(f"Count: {count}")
        count += 1
# Don't forget to update the condition!
```

```
# User input validation
password = ""
while password != "secret":
        password = input("Enter password: ")
print("Access granted!")
```

Loop Control Keywords

- break Statement

```
# Exit the loop immediately
for i in range(10):
        if i == 5:
                break
        print(i)
# Output: 0, 1, 2, 3, 4

# Finding first even number
numbers = [1, 3, 7, 4, 9, 2]
for num in numbers:
        if num % 2 == 0:
                print(f"First even number: {num}")
                break
```

Loop Control Keywords

- continue Statement

```
# Skip current iteration
for i in range(10):
        if i % 2 == 0:                  # Skip even numbers
                continue
        print(i)
# Output: 1, 3, 5, 7, 9

# Process only valid data
data = [1, -2, 3, -4, 5, -6]
for value in data:
        if value < 0:
                continue                # Skip negative values
        print(f"Processing: {value}")
```

Loop Control Keywords

- pass Statement

```
# Placeholder - does nothing
for i in range(5):
        if i == 2:
                pass            # TODO: implement special logic later
        else:
                print(i)

# Empty function placeholder
def future_feature():
        pass                    # Will implement this later

# Empty class placeholder
class MyClass:
        pass
```

Nested Loops

- Example: Multiplication

```
# Simple nested loops
for i in range(1, 4):                    # Outer loop
        for j in range(1, 4):            # Inner loop
                print(f"{i} x {j} = {i*j}")
        print("-" * 10)                  # Separator
```

Nested Loops

- Pattern Printing

```
# Print triangle pattern 列印三角形圖案
rows = 5
for i in range(rows):
        for j in range(i + 1):
                print("*", end="")
        print() # New line 換行

# Output:
# *
# **
# ***
# ****
# *****
```

Loop Optimization

- Tips for Better Performance
  1. Avoid repeated calculations

```
# Bad
for i in range(1000000):
        result = expensive_function()          # Called every iteration
        print(i * result)

# Good
result = expensive_function()                  # Called only once
for i in range(1000000):
        print(i * result)
```

From C to Python: Control Structure Comparison

- C-style thinking vs Python-style thinking

```
# ===== C-style approach =====
def grade_classifier_c_style(score):
"""C-style nested if-else"""
    if score >= 90:
        if score <= 100:
            return "A"
        else:
            return "Invalid"
    else:
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:
                    if score >= 0:
                        return "F"
                    else:
                        return "Invalid"
```

```
# ===== Python-style approach =====
def grade_classifier_python_style(score):
"""Pythonic approach with guard clauses"""
    # Input validation first
    if not (0 <= score <= 100):
        return "Invalid"

    # Linear condition checking
    if score >= 90: return "A"
    if score >= 80: return "B"
    if score >= 70: return "C"
    if score >= 60: return "D"
    return "F"
```

From C to Python: Control Structure Comparison

- Python-style thinking vs Advanced Python approach

```python
# ===== Python-style approach =====
def grade_classifier_python_style(score):
"""Pythonic approach with guard clauses"""
    # Input validation first
    if not (0 <= score <= 100):
        return "Invalid"

    # Linear condition checking
    if score >= 90: return "A"
    if score >= 80: return "B"
    if score >= 70: return "C"
    if score >= 60: return "D"
    return "F"
```

```python
# ===== Advanced Python approach =====
def grade_classifier_advanced(score):
"""Using lookup table for O(1) performance"""
    if not (0 <= score <= 100):
        return "Invalid"

    # Boundary-based lookup
    grade_boundaries = [(90, "A"), (80, "B"), (70, "C"),
(60, "D")]
    for boundary, grade in grade_boundaries:
        if score >= boundary:
            return grade
    return "F"
```

From C to Python: Control Structure Comparison

- Performance comparison

```
# Performance comparison
import time

def benchmark_approaches():
    test_scores = [95, 85, 75, 65, 55, 105, -5] * 1000

    approaches = [
        ("C-style", grade_classifier_c_style),
        ("Python-style", grade_classifier_python_style),
        ("Advanced", grade_classifier_advanced)
    ]

    for name, func in approaches:
        start = time.perf_counter()
        for score in test_scores:
            func(score)
        end = time.perf_counter()
        print(f"{name:12}: {end - start:.6f} seconds")

benchmark_approaches()
```

Summary

- Python Basic Syntax Python
  - Indentation rules and PEP 8
  - Variable naming and data types
  - Operators and type conversion
- Conditional Control
  - if, elif, else statements
  - Nested conditions and ternary operator
  - Short-circuit evaluation
- Loop Control
  - for and while loops
  - break, continue, pass keywords
  - Loop optimization techniques